SoC Compute Cluster — Quick Start

CS4248

This is a short quick-start guide for the NUS SoC compute cluster. It walks through connecting, setting up Conda, and submitting a test Slurm job so you can confirm everything works before running your own project.

Replace `<soc_username>` with your own SoC username everywhere it appears below.

For full explanations, troubleshooting, and the official references, see the longer guide at https://github.com/isaacNingLee/Unofficial_SoC_Cluster_Guide.

1 Prerequisites

Before continuing, complete the official cluster access steps, which include creating your SoC account. You will also need an SSH client and StuJump configured if you are connecting from outside the SoC network.

Official references

Always follow the official documentation when it differs from this guide:

- Accessing the compute cluster
- Getting started with the compute cluster
- Compute cluster hardware
- Job priority
- StuJump quick guide

2 Connect to the cluster

If you are outside the SoC network, connect through StuJump:

```
ssh -J <soc_username>@stujump.comp.nus.edu.sg <soc_username>@xlogin.comp.nus.edu.sg
```

If you are inside an SoC building, you can connect directly:

```
ssh <soc_username>@xlogin.comp.nus.edu.sg
```

You are now on the `xlogin` login node. Use it only to download files and submit jobs — never for installing software or running compute-heavy work directly.

Important: connect using a plain terminal SSH client only. Do not use VSCode's Remote-SSH extension (or similar IDE-integrated SSH tools) to connect to the cluster — this is not allowed.

3 Download this guide

Clone this repository so you have the job template and the dummy test script available on the cluster:

```
git clone https://github.com/isaacNingLee/Unofficial_SoC_Cluster_Guide.git
cd Unofficial_SoC_Cluster_Guide
```

4 Request a compute node

Reserve a node with Slurm, then start an interactive shell on it:

```
# Reserve a node for 60 minutes. The terminal may wait here until one is free.
salloc -t 60

# Start an interactive shell on the reserved node.
srun --pty bash
```

Your prompt changes to show the compute-node hostname once `srun` starts. Run the Miniconda and Conda environment steps below inside this shell, not on `xlogin`.

Important: installing or configuring Conda directly on `xlogin` can fail (for example with `CondaMemoryError`) because the login node has very limited resources. Always do this on the compute node you just reserved.

Submitting the test job later with `sbatch` is different: it only queues the job for Slurm to run, so it does not need an interactive shell. `xlogin` is fine for that step.

5 Install Miniconda (first time only)

Skip this section if you already have Conda set up.

```
cd "$HOME"
curl -fLO https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p "$HOME/miniconda3"
source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda init bash
source ~/.bashrc
```

Return to the cloned repository before continuing:

```
cd "$HOME/Unofficial_SoC_Cluster_Guide"
```

6 Create a Conda environment

```
conda create -n myenv python=3.11
conda activate myenv
conda install numpy pandas
```

Use `conda env list` to see all environments, and `conda list` to see packages installed in the active one.

7 Submit a test job

Copy the job template and point it at the included dummy pipeline script, which deliberately runs for about two minutes so you can watch it in the queue instead of it finishing instantly:

```
cp job_template.sbatch job.sbatch

sed -i "s/YOUR_JOB_NAME/test_job/" job.sbatch
sed -i "s/YOUR_CONDA_ENV/myenv/" job.sbatch
sed -i "s#path/to/your_pipeline_script.sh#dummy_pipeline.sh#" job.sbatch
```

`sbatch` is the command that submits a job to Slurm. Submit it from the repository root:

```
sbatch job.sbatch
```

It prints a line like:

```
Submitted batch job 917655
```

Copy that number and store it in a variable, so the commands below do not need it retyped (replace `917655` with your own number):

```
JOB_ID=917655
```

Keep using this same shell for the rest of this guide — `JOB_ID` is only remembered within it.

8 Confirm the job is running

Check the queue. You should see your job as `PD` (pending) and then `R` (running):

```
squeue --me
```

Once the job shows as running, watch its log fill in live:

```
tail -f "logs/slurm/test_job-${JOB_ID}.out"
```

Press `Ctrl+C` to stop watching — this only stops `tail`, not your job.

9 Confirm the job finished

After about two minutes, check the queue again; the job should no longer appear:

```
squeue --me
```

Then read the full log:

```
cat "logs/slurm/test_job-${JOB_ID}.out"
```

You should see start/end timestamps, the step counter reaching `12/12`, and `nvidia-smi` output if a GPU was allocated. If you see all of that, your environment and job-submission workflow are working end to end.

10 Clean up

When you are done, leave the compute shell and release the allocation:

```
exit # leaves the compute-node shell started by srun
exit # releases the salloc allocation
```

11 Next steps

Replace `dummy_pipeline.sh` and the Conda environment/packages with your actual project, following the full guide at https://github.com/isaacNingLee/Unofficial_SoC_Cluster_Guide for details on storage locations, job priority, and requesting specific GPUs.